



AI-Powered Intelligent Job Search and Career System

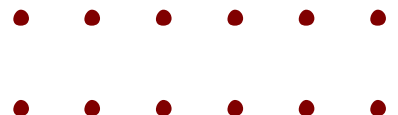
Data Science and AI Project

Milestone 6 Report

Group 4

Prepared By:

Gaurav Kumar (22f1001105)
Dev Gupta (22f2000888)
Abhinav Ohri (24f1002064)
Pranav N (22f2000117)
Praveena N (22f3001454)





Index

SN	Particulars	Page No
1	Abstract	2
2	Introduction	2
3	Literature Review	4
4	Dataset and Methodology	7
5	Model Development and Configuration Tuning	11
6	Evaluation and Analysis	17
7	Deployment and Documentation	21
8	Conclusion and Future Work	25
9	References	26
10	Team Consent	27
11	Appendix	28

1. Abstract

A candidate searching for their next role works across three disconnected surfaces: a resume maintained by hand, career-guidance sites that describe occupations in generic terms, and job boards that match on keywords. Nothing carries context from one to the next.

This project builds DiscoverMyRole, an end-to-end system that closes that gap. It converts an uploaded resume into a schema-validated candidate profile, maps that profile onto occupations in the ESCO v1.2.1 taxonomy, and uses the resulting role context to discover and rank live job postings — in one pass, from one application.

The system performs no gradient-based training. Every model is pretrained and used either zero-shot through a schema-constrained prompt or as a frozen encoder. This was a design decision, not a limitation discovered late: the project has no labelled training data of the kind a supervised formulation would require. The entire improvement surface is therefore configuration — prompts, output schemas, retrieval parameters, and scoring weights — selected by exhaustive sweeps over small discrete grids and scored by deterministic, rule-based validators.

Evaluation was conducted against 86 hand-annotated resumes stratified two per category across 43 categories, 58 of them fully source-verified against their original PDFs, with a 52-resume held-out split. Resume Parsing meets its 0.75 F1 acceptance threshold in all six sections (Contact 0.98, Experience 0.98, Education 0.92, Projects 0.88, Skills 0.82, Certifications 0.75). Career Recommendation achieves MRR 0.6121, Hit@7 0.8488, and Hit@20 0.9186 with zero unrecoverable failures across 86 profiles, and generalises to the held-out split at MRR 0.6359. Job Matching reaches Precision@5 0.8486 and NDCG@10 0.9795 at 226 ms.

Two results did not clear their acceptance bar and are reported as failing rather than absorbed into a favourable summary. The batched judge stage returns no passing configuration across six prompt variants; the root cause is identified as token overflow rather than prompt wording. Crawl success in the fallback discovery path remains 0.50. Separately, the deterministic re-ranking step's headline Hit@5 gain is one profile out of 86 ($p = 1.00$ on a paired exact test), so the claim that it improves ranking has been withdrawn in favour of the weaker and defensible claim that it does not degrade the retrieval ordering, which the rule it replaced did.

The final system is deployed on AWS EC2 and documented for reproduction. The Resume Parsing and Career Recommendation path is ready to be used as decision support. It is not ready to be used as an automated decision, and it is not presented as one.

Keywords

Retrieval-augmented generation; ESCO occupational taxonomy; vector search; pgvector; hybrid ranking; resume parsing; vision-language models; deterministic re-ranking; explainable recommendation.

2. Introduction

2.1 Problem statement

Career decisions are made with poor information. A job seeker knows their own history but not how it maps onto the labour market's own vocabulary; occupational taxonomies describe roles precisely but have no view of any individual; job boards see postings but match on surface keywords that miss paraphrase and related skills. The work of joining these three views is left entirely to the candidate, done manually, and rarely done well.

The specific difficulty is that each of the three surfaces speaks a different language. A resume says "built dashboards in Power BI and cleaned data pipelines". A taxonomy says "data analyst — essential skills: data visualisation, data cleansing, statistical analysis". A job posting says "3+ years BI experience, SQL required". Matching across those registers is a semantic problem, not a keyword problem.

2.2 Objective

Build a single application that takes a resume as input and produces two grounded, explained outputs: a ranked set of occupational roles the candidate plausibly fits, and a ranked shortlist of live job postings seeded from those roles. Every output must carry its evidence, which skills matched, which are missing, and which retrieved record supports the claim, rather than an unexplained score.

2.3 Scope and constraints

Three constraints shaped every architectural decision in this project and are stated here because they explain choices that would otherwise look arbitrary.

- No labelled training data. There is no corpus of (candidate profile → correct occupation) pairs at meaningful volume per class, and 3,039 occupations would require thousands of such pairs. This ruled out supervised classification and made retrieval the only viable formulation.
- No GPU, and free-tier API quota. All large models are called over hosted APIs; the only local model is a 110M-parameter CPU encoder. The binding cost constraint throughout was never money but request quota — roughly 1,500 requests per day and 15 per minute. This forced the local encoder, the offline index build, the batched judge design, and the zero-LLM extraction stage.
- Privacy by construction. Email addresses and telephone numbers are never extracted or persisted; locations retain locality only; original uploads and rendered page images are transient. Retained sensitive fields are encrypted at field level.

2.4 System summary

The system is organised as modules connected through one shared candidate profile, all running inside a single FastAPI application over one centralised Supabase PostgreSQL database with the pgvector extension, fronted by a React and TypeScript interface.

Module	Receives	Produces
Resume Parsing	Uploaded resume (PDF / DOCX / image)	Validated Candidate Profile JSON
Career Recommendation	Candidate Profile JSON	Top-N ranked ESCO occupations with grounded explanations
Job Discovery and Matching	Candidate Profile + recommended roles	Ranked shortlist of postings with match score and rationale
Career Report	Profile + recommendations + shortlist	HTML and PDF career report
CV Review	Stored candidate profile	Recruiter-style critical findings (read-only)
Feedback	Rating, reason, optional comment, optional profile ID	Stored feedback record; aggregate summary counts

2.5 Milestone progression

Milestone	Focus	Principal outcome
1	Problem framing and literature review	Three-module architecture and the retrieval-over-classification decision
2	Dataset preparation and validation	ESCO v1.2.1 prepared as a retrieval knowledge base; resume and job-posting corpora selected and validated
3	Architecture finalisation	Seven-stage LangGraph Job Discovery pipeline; retrieval, re-ranking, prompt, schema, and API contracts specified

4	Prototype implementation and configuration tuning	All three modules running end to end; the no-gradient-training framing established; configuration sweeps recorded
5	Evaluation and integration	Field-level and ranking evaluation with confidence intervals; migration to Supabase pgvector; two components reported as failing
6	Deployment and documentation	AWS EC2 deployment; Feedback module as the value-added enhancement; full reproducibility documentation

2.6 Report structure

Sections 3 through 7 follow the milestone sequence: literature review, dataset and methodology, model development and configuration tuning, evaluation and analysis, and deployment and documentation. Section 8 concludes and sets out future work. Section 9 lists references and Section 10 provides supporting appendices.

3. Literature Review

This section corresponds to Milestone 1. The full literature survey conducted at that milestone is not reproduced here in the detail of the original submission; the summary below records the conclusions that governed all subsequent design work.

3.1 Conclusions carried forward from the survey

Four findings from the Milestone 1 survey determined the architecture and are visible in every subsequent decision.

Resume parsing is a document-understanding problem, not a text problem

The working corpus is entirely single-page, image-only PDFs with no text layer. This makes the task multimodal by necessity. The alternatives considered, LayoutLM and LayoutLMv3, Donut, spaCy or BERT-based NER over OCR text, and regex or rule-based parsers, were rejected because each requires task-specific fine-tuning, field-level labels, a separate OCR stage, or brittle template-specific rules. Encoder-only architectures were rejected on a structural ground: they label input tokens rather than generate new ones, and so cannot produce a variable-length nested JSON document as output.

Retrieval is preferable to classification when the label space is large and unlabelled

A 3,039-class supervised classifier over occupations would require thousands of labelled candidate profiles distributed across every class to avoid catastrophic imbalance. Retrieval-augmented generation requires no labelled candidate-to-occupation pairs at all — only the taxonomy itself, which is already curated and complete. This single decision is the foundation of the Career Recommendation module.

Explainability is a functional requirement in career guidance, not a nicety

A recommendation a candidate cannot interrogate is a recommendation they cannot act on. A fine-tuned model's prediction is an opaque function of its weights; retrieval keeps every recommendation traceable to a specific retrieved record, the exact occupation URI, its essential skills, and inspectable deterministic re-ranking logic, all of which can be shown to the user as evidence.

Hybrid lexical and semantic ranking outperforms either alone in job matching

Lexical methods such as BM25 capture exact tool and certification names that embeddings blur; dense embeddings capture equivalent skills phrased differently, which lexical methods miss

entirely. The literature favours combining both. It is worth recording that our own ablation at Milestone 5 could not confirm this on our evaluation pool, a point returned to in Section 6.

3.2 Problem context and current practice

The Milestone 1 survey opened by establishing why the problem is worth solving before surveying how it is currently handled. LinkedIn's 2026 workforce research was cited as showing that more than half of people globally are looking for a new role, that 65% of people say finding a job has become more challenging, and that 93% of recruiters plan to increase their use of AI in 2026, evidence that both sides of the hiring process are becoming more AI-mediated, not less. The survey identified three recurring difficulties for candidates operating across LinkedIn, Naukri, Indeed, Glassdoor and company career pages: discovery overload (too many listings to judge fit manually), random applications (applying broadly without knowing which roles are suitable), and lack of feedback (no clear signal on why a profile is or is not a match).

Commercial tools, by function

- **Job discovery** — handled by LinkedIn, Naukri, Indeed, Glassdoor and company career pages through search filters, saved alerts and recommendation widgets, but candidates still read and judge each listing by hand.
- **Career guidance** — delivered through static guides, skill taxonomies and simple recommender tools that suggest roles generically, by job title or broad skill category, rather than from an individual's actual education, projects and experience.
- **Resume and application support** — resume builders and general-purpose AI assistants can draft, tailor or generate cover letters, but this project treats tailoring and cover-letter generation as optional good-to-have features rather than core scope.

3.3 Academic approaches surveyed

Four bodies of academic work were reviewed, one per pipeline stage, and each is summarised here with the specific architectural conclusion it produced.

Resume parsing as information extraction

Resume parsing is typically framed as an information-extraction task. Transformer encoders such as BERT learn contextual meaning from text and can be fine-tuned for entity extraction but assume a usable text layer. For resumes with complex layouts, layout-aware models such as LayoutLM and its successor LayoutLMv3 were reviewed because they jointly model text and page layout, which matters when a resume's visual structure carries meaning (columns, headers, tables). OCR-free document-understanding transformers such as Donut were also reviewed as an alternative that skips a separate OCR stage entirely. All of these were noted as requiring task-specific fine-tuning and field-level labelled data that this project does not have, which is the reasoning trace that led to the zero-shot, schema-constrained multimodal extraction approach used in the final system.

Resume classification and profile understanding

Work such as ResumeAtlas was reviewed to understand how large resume datasets and LLM-based methods classify resumes and construct candidate profiles, and it explicitly surfaces the challenges of privacy, inconsistent resume formats and uneven data quality — challenges this project also encountered in its own 8,905-document corpus (Section 4.7).

Job matching as semantic retrieval

Traditional job matching depends on keyword overlap between a resume and a posting, which misses paraphrase and related skills expressed in different vocabulary. Semantic embedding models such as Sentence-BERT were reviewed as the standard remedy, comparing resumes and postings by meaning rather than surface form. Person-job fit models such as those explored by Qin

et al. and the ConFit line of work were reviewed for their use of neural and contrastive-learning methods to model resume-job compatibility directly, reinforcing the survey's conclusion that hybrid lexical plus semantic ranking is the appropriate design for Job Matching.

Career recommendation via taxonomies and knowledge graphs

Career recommendation work reviewed included recommender systems, knowledge-graph approaches, and skill-role taxonomies; research such as Dave et al. studies the relationships between jobs and skills directly. Public occupational taxonomies — principally O*NET and ESCO — were compared as the reference structure connecting skills, occupations and career paths. ESCO was selected over O*NET because it ships direct, machine-readable occupation-skill relation files (occupationSkillRelations_en.csv) that make retrieval-ready ingestion straightforward, whereas O*NET's data model requires more assembly to reach the same structure. India's National Classification of Occupations (NCO) was also considered as a India-specific alternative but was not adopted as the primary taxonomy because it lacks ESCO's machine-readable skill-relation files and its English-language occupation-skill coverage at the scale ESCO provides; the survey noted this as a limitation to be evaluated empirically rather than assumed, which is why local and non-European title variants are evaluated separately in this project's methodology (Section 4.6).

3.4 Metrics and existing definitions of success

The survey found that existing work measures success differently at each stage of the pipeline and grouped these into quantitative and qualitative metrics.

- **Precision, Recall and F1-score:** applied to resume extraction; compares extracted structured fields against manually checked ground truth.
- **Top-K Accuracy and Mean Reciprocal Rank (MRR):** applied to career-path prediction; Top-K checks whether a suitable role appears in the top suggestions, MRR checks how high the first correct recommendation ranks.
- **LLM-as-judge scoring:** applied to career recommendations; a separate language model rates relevance, clarity, personalisation, factual accuracy and role fit on a 1-5 scale.
- **Human review:** used to check that tailored resumes and cover letters are specific, professional and grounded only in the candidate's real profile, since generated text must not include fabricated skills or unsupported claims.

Academic systems were found to rely on these proxy metrics because real hiring outcomes are typically unavailable, while commercial platforms instead measure user and business outcomes such as application conversion, interview callback rate and time-to-hire. This project adopted the academic framing: success is defined by accurate resume parsing, complete candidate-profile creation, relevant career recommendations and useful job ranking, evaluated with the proxy metrics above rather than with hiring-outcome data this project does not have access to.

3.5 Gaps identified and this project's positioning

The survey's central conclusion was that current tools are disconnected rather than individually weak: a candidate must move between separate platforms for job search, career guidance, resume support and application preparation, manually carrying context between each. Five specific gaps were identified as this project's opportunity.

- **Job discovery limited to major boards:** candidates search only a handful of popular platforms, which surface many jobs but give no help identifying which ones fit.
- **A fragmented workflow:** resume parsing, career recommendation and job search are handled by separate tools with no shared state, so the candidate re-enters information at every step.
- **Generic career guidance:** existing recommenders suggest by job title or broad category rather than from a candidate's actual education, projects, skills and experience.

- **Limited profile-based feedback:** candidates rarely learn why a job is or is not suitable for them.
- **Separate application support:** resume tailoring, ATS-style scoring and cover-letter generation exist as standalone tools rather than as part of one connected flow.

This project's contribution, as framed at Milestone 1, is not a claim of beating academic benchmarks. It is a practical system contribution: integrating resume parsing, profile creation, career recommendation and job ranking into one connected workflow; grounding every recommendation in the candidate's actual structured profile; making job ranking explainable through matched skills, experience and role similarity; and, where time allows, adding resume tailoring, ATS scoring and cover-letter generation as optional, profile-grounded features. Sections 3.1 above and 4 through 8 that follow record how far that positioning survived contact with real data, real API limits, and measurement.

4. Dataset and Methodology

This section corresponds to Milestones 2 and 3. Dataset preparation and validation were completed at Milestone 2; the architecture, retrieval design, prompt engineering, and API contracts were finalised at Milestone 3.

4.1 Datasets

Dataset	Version and size	Role
Resume corpus	Kaggle hadikp/resume-data-pdf — 8,905 single-page image-only PDFs; 86-record hand-annotated gold subset	Development and evaluation of Resume Parsing
Occupation taxonomy	ESCO v1.2.1 English — 3,039 occupations, 13,939 cleaned skills, 126,051 valid occupation-skill relations	Retrieval knowledge base for Career Recommendation, indexed in full
Career Recommendation labels	134 category-to-occupation mappings across 43 categories	Ground truth for retrieval and ranking metrics — a proxy, not direct occupation labels
Candidate Job Role Dataset	Kaggle — 1,000 profiles; 200-profile stratified development sample	Query side of the ranking experiments
LinkedIn Job Postings Dataset	Kaggle — 123,849 postings, 24,428 companies, 72,521 titles	Offline document pool for ranking experiments
Job Matching golden set	86 candidates × 10-job pool	Ranking evaluation with real BM25 and embedding scores
Live discovery store	Populated at runtime, deduplicated by hash of the normalised posting URL	Runtime postings; not used for offline experiments

Every prepared ESCO occupation has linked skills, and the occupation set covers all ten ISCO major groups. ESCO is used as a general occupational taxonomy, not as a supervised training dataset.

4.2 Splits

Module	Split	Rationale
Resume Parsing	Development 34, held-out test 52, training 0	Gold set of 86 records stratified two per category across all 43 canonical categories. Near-duplicate groups are confined to a single split so a visually similar resume cannot straddle the boundary. Random seed 42.
Career Recommendation	Not split (index); inherits the resume split for evaluation	The ESCO occupation set is the complete retrieval knowledge base. Withholding occupations would degrade the system rather than create a meaningful held-out condition, since a candidate cannot be matched to an occupation absent from the index.

Job Discovery and Matching	60 / 20 / 20 on the 200-profile development sample: 120 / 40 / 40 candidates	Candidate IDs are divided before candidate-job pair construction, so every pair belonging to one candidate stays in the same split. 60/20/20 was chosen over 80/10/10 because the latter would leave only 20 candidates per evaluation split — too few for stable ranking estimates.
----------------------------	--	--

No k-fold cross-validation is used. The gold set holds two resumes per category, so a fold would place a single resume per category, producing unstable per-fold estimates. A single stratified development and held-out split is used instead.

4.3 Ground-truth construction and its honest limits

The resume corpus carries a resume category rather than an ESCO occupation label. Each of the 43 categories was therefore mapped to a set of acceptable ESCO occupations — 134 mappings in total, generated by keyword matching against ESCO titles and then reviewed by hand. A recommendation counts as correct when it matches any occupation in the set for that resume's category.

This mapping is a proxy and is described as such throughout. Several categories map to overlapping occupation sets, which caps achievable precision, and four categories initially returned no match. It supports rank-position measures such as MRR and Hit Rate rather than set-coverage measures.

At Milestone 5 the gold annotations themselves were corrected: 58 of the 86 records were manually verified against their source PDFs, with corrections stored as versioned overlays so the original annotations remain auditable. The corrections were not cosmetic — they changed what the evaluation measures. One DevOps record had empty gold skills despite visibly listing Kibana, Prometheus, Datadog, CI/CD, Jenkins, Terraform, AWS, Azure, GCP, Docker and Kubernetes; one web-design record was linked to the wrong reference profile entirely and was replaced with the profile belonging to its source PDF.

4.4 Preprocessing

Dataset	Format	Preprocessing
Resume corpus	Single-page image-only PDF; no extractable text layer	PyMuPDF renders each page to an image. A 100-character-per-page text-layer probe selects the text path only when one exists; otherwise the vision path is used. One page per model call; per-page JSON merged afterwards; skills deduplicated case-insensitively.
ESCO taxonomy	Three relational CSVs	Semantic atomic chunking — one chunk equals one complete occupation profile, concatenating preferred label, alternative labels, description, essential skills, and optional skills. Embedded once offline and written to the pgvector index.
Job postings	Structured CSV plus live HTML	Structured fields read from schema.org JobPosting JSON-LD or page metadata with zero LLM calls. Postings split into RAG chunks by section header; postings without recognisable headers fall back to a single chunk.

Traditional sliding-window chunking was rejected for ESCO because the taxonomy is a structured, atomic occupational classification rather than free-form text. One chunk per occupation ensures the embedding vector holistically represents the full context of a role rather than fragmenting critical skill information across multiple entries.

Augmentation was used for robustness testing only, never to expand a training set, since there is none. A separate robustness set was generated using small image rotations, moderate Gaussian

blur, JPEG compression, contrast reduction, resolution reduction, and light image noise; each augmented document inherits the annotation and split of its source, so a transformed copy cannot leak across the evaluation boundary.

4.5 Feature representation

There is no hand-engineered feature vector anywhere in this system. Representation learning is entirely internal to the pretrained models.

- **Resume Parsing:** the model encodes the page image internally; the prompt and schema are the only engineering. No separate encoder is built.
- **Career Recommendation:** the candidate profile is flattened into a semantic query string and embedded to 768 dimensions. Essential and optional skills are stored as separate metadata fields on each chunk rather than concatenated, so an essential-skill match can be weighted above an optional one (1.0 against 0.5).
- **Job Matching:** each job is represented both as structured relational columns for exact SQL filtering and as one job-level 768-dimensional embedding plus multiple chunk-level embeddings for RAG retrieval at judge time.

4.6 Methodology

Retrieval design

Retrieval is split into a broad initial vector search followed by a deterministic refinement step, balancing recall against precision. An initial Top-K of 20 reduces the risk of prematurely discarding hyper-specialised or unusually titled roles; the wide window is justified empirically, since Hit@20 of 0.9186 confirms the correct occupation is almost always inside it. A deterministic Python re-ranking stage then applies hard-requirement exclusions followed by a blended score combining cosine similarity with normalised skill overlap, reducing to a final Top-N passed to the explanation model.

This re-ranking is intentionally rule-based rather than a second embedding or classification pass, so that occupation order is reproducible, inspectable, and requires no additional model call.

Grounding and hallucination control

The explanation model operates under hard constraints rather than instructions alone. It may only discuss occupations present in the retrieved set and may only name skill gaps drawn from that occupation's own ESCO skill list. Every returned occupation URI is validated against the retrieved set and a non-matching entry is dropped before the response leaves the server. Confidence bands are derived from the deterministic evidence — five matched skills yield high, one yields low — not asserted by the model.

Three mode-specific instruction blocks are selected at run time: the standard grounded-explanation instruction; an instruction used when the candidate profile lists no skills at all, directing assessment onto job titles and experience and requiring the response to say so; and an instruction used when listed skills produce no overlap with any retrieved occupation, requiring low confidence and an explicit statement that the match is similarity-only. These paths exist so that a sparse or unusual profile produces a qualified answer rather than an empty one.

Token minimisation as a design principle

The guiding principle for Job Discovery was: never call a model where a deterministic method achieves comparable results, and where a call is unavoidable, batch it across every item that needs it rather than issuing one call per item. A naive per-item design would issue one extraction call per discovered job and one judge call per shortlisted job — an estimated 45 or more calls per run. Making extraction deterministic and batching the judge reduces this to exactly two calls per run regardless of how many jobs are found.

Evaluation protocol

Each component follows the same loop: the configuration surface and metrics are fixed before the sweep runs, alternative configurations are run with everything else held constant, each is scored by a deterministic validator into one weighted composite, the highest composite is selected automatically, and the winner is re-run against an explicit baseline. All grading is deterministic and rule-based rather than model-graded, specifically so the evaluation itself stays reproducible and to avoid a second model call per evaluated output.

4.7 Exploratory Data Analysis

Milestone 2 profiled all three prepared datasets before any modelling work began. The findings below governed the preprocessing and configuration decisions summarised in Sections 4.4 and 5.4.

4.7.1 Resume Parsing corpus

EDA item	Result
Total resumes	8,905
Pages per resume	1 for all files
Raw category folders → canonical categories	97 → 43
Corrupt files	0
Image-only PDFs	8,905 — 100%
Files with extractable text layer	0
Exact duplicate files (MD5)	0
Perceptual near-duplicate groups	6,611
Files in multi-file near-duplicate groups	3,291
Excess copies beyond one representative per group	2,294
Largest category	Human Resources — 358 resumes
Smallest category	BPO — 29 resumes
Category imbalance ratio	≈ 12.3 : 1
Median image resolution	962 × 1,600 pixels
Median file size	231 KB
Median OCR-estimated length (200-resume sample)	≈ 374 words / 2,699 characters / ≈ 500 tokens
95th-percentile token length	≈ 1,655 tokens

The imbalance ratio of approximately 12.3:1 between the largest category (Human Resources) and the smallest (BPO) is the reason the gold evaluation set was sampled two resumes per canonical category rather than proportionally to corpus size — proportional sampling would have under-represented 42 of the 43 categories. The 95th-percentile length of roughly 1,655 tokens confirmed that almost every resume could be processed in a single model call without document chunking, which shaped the one-page-per-call design used in the final parsing pipeline (Section 4.4).

Common skills across categories: Communication (11 resumes in the annotated sample), SQL (9), Leadership and Project Management and Python (7 each), SQL Server and Microsoft Office and AutoCAD (6 each). Because these terms recur across unrelated occupation categories, occupation cannot be identified reliably from skill overlap alone — the downstream Career Recommendation module therefore combines skills with education, experience, projects and job titles rather than relying on skills in isolation.

Gold-set field group	Records containing the field (of 86)
Skills	86
Job titles	85
Experience	80
Education	69
Certifications	27
Projects	5

Projects and certifications are comparatively sparse in the source corpus, which is why Section 6.2's field-level evaluation reports per-field results rather than a single pooled score — a pooled figure would let the well-populated Contact and Experience fields mask weaker performance on the sparse ones.

4.7.2 Career Recommendation taxonomy (ESCO)

Metric	Value
Clean occupations	3,039
Clean skills	13,939
Skills used in occupation relations	13,475
Occupation–skill relations	126,051
Essential relations	67,600
Optional relations	58,451
Unused skills (present in master file, not linked)	464
Occupations with zero linked skills	0
Average / median / min / max skills per occupation	41.48 / 37 / 7 / 178
ISCO major group	Number of occupations
Professionals	868
Technicians and associate professionals	646
Craft and related trades workers	394
Managers	350
Plant and machine operators and assemblers	347
Services and sales workers	204
Elementary occupations	76
Clerical support workers	89
Skilled agricultural, forestry and fishery workers	44
Armed forces occupations	21

All ten ISCO major groups are represented, which supports generalisation beyond software and technology roles even though the candidate-side evaluation data is IT-skewed (Section 4.7.3). No occupation has zero linked skills, so every occupation has enough content to build an embedding context.

Common-skill analysis: skills such as 'Use different communication channels' (340 occupations), 'Create solutions to problems' (339), 'Manage staff' (334), 'Keep records of work progress' (321) and 'Manage budgets' (319) are linked to hundreds of occupations and carry little discriminatory value on their own.

Rare-skill analysis: 2,885 skills are linked to only one occupation, of which 2,025 are essential skills for that occupation — examples include specialised teaching principles, technical equipment knowledge and occupation-specific tools. Rare skills provide much stronger evidence for a specific occupation than common ones.

These two findings are the direct basis for the skill-specificity weighting described in Section 4.5: common skills are retained but down-weighted, rare skills are given higher specificity and placed earlier in the embedding context, and essential skills are prioritised ahead of optional skills.

4.7.3 Job Discovery and Matching datasets

Metric	Candidate Job Role Dataset	LinkedIn Job Postings Dataset
Raw records	1,000	123,849
Unique values of note	98 skill combinations, 17 qualifications, 4 experience levels, 22 job roles	24,428 companies, 72,521 job titles, 8,526 locations

Most common category	Mobile Developer (75 candidates); Bachelors in CS (349); Mid experience (401)	Full-time postings dominate every experience band
Missing values / exact duplicates	0 / 0	— / 0 exact duplicate rows
Geographic coverage	Not available (no location field)	Only 742 of 123,849 ($\approx 0.60\%$) explicitly mention India; mostly U.S.
Time window	—	24 Mar 2024 – 20 Apr 2024 (26 days)

72,521 unique job titles across 123,849 postings, with 60,922 titles occurring only once, confirmed that raw job titles cannot be treated as stable classes — this is the reason free-text titles are fuzzy-matched to a controlled role vocabulary rather than compared directly (Section 4.6).

A preliminary ESCO alignment test using exact and token-based fuzzy matching found that 68.2% of the 22 candidate job roles and 92% of the 50 most frequent LinkedIn job titles received an ESCO match. These figures were treated throughout as coverage estimates rather than guaranteed mapping accuracy, since low-confidence fuzzy matches were reviewed rather than accepted automatically.

Skill frequency by role: frontend roles emphasise React, HTML, CSS and JavaScript; backend and full-stack Java roles emphasise Java, SQL and AWS; Data Analyst, Data Scientist and AIML roles share Python, SQL, machine learning and TensorFlow; DevOps and Kubernetes roles emphasise Docker, Kubernetes and AWS; HR and Marketing carry more domain-specific skills such as recruitment. This confirmed that job matching needed a combination of semantic similarity, title alignment and experience fit rather than individual skill keywords alone.

Salary was found to increase with experience on average but with wide variation and extreme outliers at both the entry and mid-senior levels, so salary was kept as an optional, log-transformed signal rather than a core relevance input. Full-time postings were found to dominate every experience category, so employment type is applied only when a candidate states a preference, preventing the base rate of full-time listings from inflating relevance automatically.

4.7.4 Data validation and cleaning procedure

Dataset	Validation and cleaning performed
Resume corpus	Every PDF opened and rendered with PyMuPDF (0 corrupt files); text-layer probe confirmed 0 of 8,905 files had extractable text; 97 folder labels lower-cased and mapped through a manually maintained alias table to 43 canonical categories, merging only spelling, formatting, abbreviation or plurality variants (e.g. ‘Data Science’ / ‘DataScience’ / ‘data science’) while keeping distinct-but-related categories such as Designer and Web Designing separate; MD5 hashing for exact duplicates (0 found); perceptual hashing (Hamming distance ≤ 6) for near-duplicates, grouped into dup_group rather than deleted; no documents discarded during category mapping.
ESCO taxonomy	occupations_en.csv, skills_en.csv and occupationSkillRelations_en.csv joined on occupation and skill URIs; 4 duplicate/invalid occupation records and 21 duplicate/invalid skill records removed (3,043 $\rightarrow$ 3,039 occupations; 13,960 $\rightarrow$ 13,939 skills); relations referencing an invalid or unavailable URI excluded; essential and optional relation types retained separately; 0 of 126,051 relations removed once source records were clean.
Job Discovery and Matching data	Candidate values trimmed and standardised (e.g. ‘Entry,’ corrected to ‘Entry’); comma-separated skills split into normalised lists; job postings stripped of URLs, platform metadata, ZIP/FIPS codes, sponsorship fields and view/application counts (31 raw columns $\rightarrow$ 16 retained); experience labels on both sides mapped to a shared Entry / Junior / Mid / Senior / Unknown scale; free-text job titles matched

	to the 22 candidate roles via RapidFuzz token-based similarity with low-confidence matches left Unmapped rather than forced.
--	--

Across all three datasets the guiding validation principle was the same one applied throughout the rest of the project: merge only what is demonstrably equivalent, keep ambiguous or low-confidence mappings unmapped rather than forcing them, and record what was discarded so that every cleaning decision remains auditable. No resume documents, ESCO occupations or job postings were discarded outright during Milestone 2 preparation; parsing difficulty, category imbalance and geographic skew were documented as limitations to be handled downstream (Sections 4.2 and 6.9) rather than filtered out of the corpus.

5. Model Development and Configuration Tuning

This section corresponds to Milestone 4. Because this system trains no weights, the conventional headings of a model-development chapter are mapped onto their actual equivalents rather than skipped.

5.1 The no-training position

This system performs no gradient-based training. No model weights are initialised, updated, or saved at any point in the pipeline. Every model is pretrained and is used either zero-shot or few-shot through a prompt and an output schema, or as a frozen encoder that produces embeddings. This is a design decision with a stated reason, the project has no labelled training data of the kind these tasks would require, and not an omission. Training is unnecessary here, not impossible: a strong instruction-following multimodal model already generalises to resume extraction zero-shot, so the system avoids the cost of building, labelling for, and maintaining a custom-trained model.

5.2 Mapping conventional training activities

Conventional activity	Equivalent in this system
Trainable weights, optimiser, learning rate	None. Configurable prompts, schemas, retrieval parameters, and scoring weights chosen by exhaustive sweeps over small discrete grids
Batch size	A cost measure only: the judge scores five shortlisted jobs per call; the parser sends one page image per call
Number of epochs	No repeated passes over data; the occupation index is built in a single pass
Loss function	Deterministic rule-based validators combined into a weighted composite score, maximised by each sweep
Early stopping	A three-condition stopping criterion for prompt refinement; a fixed exhaustive grid for parameters
Checkpoint selection	Configuration-bundle selection; the winning bundle is the artefact the system runs on
Regularisation	Schema constraints, evidence grounding, URI validation, and permissive filtering
Training duration and loss curves	Index build time and evaluation runtime; the sweep response curve is the equivalent diagnostic

The stopping criterion for prompt refinement deserves particular note, because it replaces the usual reliance on visible improvement with three stated conditions: field extraction is stable, meaning repeated runs over the development split return the same fields for the same input; no schema violations occur across the development split; and no new error classes are introduced relative to the previous prompt version, checked by comparing returned JSON against the source images.

5.3 Final model configuration

Module	Component	Model
Resume Parsing	Primary parser	Gemini 3.5 Flash, vision-only at 150 DPI

Resume Parsing	Evaluated baseline	Gemma 4 31B (multimodal, open weight)
Resume Parsing	Backup and repair	Gemini 2.5 Flash-Lite
Career Recommendation	Embedding model	BAAI/bge-base-en-v1.5, 768 dim, local CPU
Career Recommendation	Vector store	Supabase PostgreSQL, pgvector, HNSW cosine index
Career Recommendation	Explanation model	Gemini 3.5 Flash-Lite
Job Discovery and Matching	Query generation	Gemini 3.5 Flash-Lite
Job Discovery and Matching	Judge	Batched judge call over the top 5
Job Discovery and Matching	Search and crawl	Azudhan job API (primary); SearXNG + Crawl4AI (fallback)
Job Discovery and Matching	Ranking	BM25 (rank_bm25) + bge-base-en-v1.5 cosine similarity

5.4 Configuration parameters and their basis

There are no hyperparameters in the training sense. The parameters below govern system behaviour and are each labelled by the basis on which they were set: empirical, where the value was selected by a measured sweep across alternatives, or heuristic, where it was set by reasoning about the task and has not yet been varied and measured. This distinction is maintained deliberately, so that a reader can see which values carry evidence and which do not.

Module	Parameter	Value	Basis
All	Temperature	0	By design — reproducibility required; sampling variety has no value in any task here
All	Embedding dimensions	768	Fixed by the encoder
All	Random seed	42	Fixed across split assignment and sampling
Resume Parsing	Provider and route	Gemini 3.5 Flash, vision-only, 150 DPI	Empirical — beat the Gemma 4 baseline on every section F1
Resume Parsing	Skill boundary rule	Preserve competency phrases; split explicit enumerations	Empirical — six prompt experiments, Skills F1 0.2268 → 0.8198
Resume Parsing	Description rule	Copy visible text without summarising	Empirical — exact copy rate and description similarity measured
Resume Parsing	Text-layer probe	100 characters per page	Heuristic — the corpus has no extractable text layer, so untested on this data
Resume Parsing	Fallback retries	1	Heuristic — cost and latency reasoning
Career Rec.	Chunk contents	Preferred/alt labels, description, essential + optional skills	Empirical in part — optional skills present for 2,857 of 3,039 occupations
Career Rec.	Vector distance	Cosine, explicit HNSW operator class	Empirical — measured against the default-distance index
Career Rec.	Skill-bonus weight	0.02	Empirical — eight values swept against MRR and Hit Rate
Career Rec.	Retrieval Top-K	20	Empirical in part — Hit@20 of 0.9186 confirms the window is wide enough
Career Rec.	Final Top-N	7	Empirical — compared at 5 and 7; Hit@7 0.8488 against Hit@5 0.8372

Career Rec.	Essential/optional skill weights	1.0 and 0.5	Heuristic — ratio not swept
Job Matching	BM25 / embedding weights	0.5 / 0.5	Empirical — six configurations compared
Job Matching	Shortlist after hybrid ranking	15	Heuristic — not varied in the sweep
Job Matching	Hybrid / judge weights	0.8 / 0.2	Heuristic — not varied in the sweep
Job Matching	RAG chunks per job at judge time	2	Heuristic — judge prompt size
Job Discovery	Source routing	Azudhan API primary; SearXNG + Crawl4AI fallback	Empirical — API path validated across domains and locations
Job Discovery	Search and crawl parameters	max_results 5, crawl limit 3, concurrency 3, timeout 8,000 ms	Empirical — five parameter sets compared

Scoring formulas

$\text{hybrid_score} = (0.5 \times \text{BM25} + 0.5 \times \text{embedding_cosine})$ # top 15 kept
 $\text{final_score} = 0.8 \times \text{hybrid_score} + 0.2 \times \text{judge_probability}$

5.5 The skill-bonus weight sweep

This sweep is the clearest example of the project's tuning methodology and its most substantive single finding, so it is presented in full.

The re-ranking step combines two signals: cosine similarity from retrieval, and exact overlap between the candidate's skills and the occupation's ESCO skill list. The question the sweep answers is how much influence the skill-overlap signal should carry relative to similarity. This was parameterised as a single skill-bonus weight and measured across eight values on the 86-record annotated set, from a pass-through at one end through to skill-dominant settings at the other.

Skill-bonus weight	MRR	Hit@1	Hit@5
0.00 (pass-through)	0.5849	0.4651	0.7791
0.02 — selected	0.5965	0.4884	0.7907
0.05	0.5866	0.4767	0.7907
0.08	0.5457	0.4070	0.7907
0.10	0.5328	0.3953	0.7674
0.15	0.4973	0.3372	0.7674
0.25	0.4866	0.3256	0.7674
0.40	0.4843	0.3256	0.7558

Table 5.1 — Skill-bonus weight sweep across eight values on the 86-record annotated set.

At weight 0.00 the step is a pass-through and reproduces the retrieval ordering exactly, which confirms the implementation is correct and establishes the floor the step must beat. Weight 0.02 improves on that floor on all three measures and was selected. Above 0.05 every measure declines monotonically.

The shape of the curve is the substantive finding, not the winning value. The endpoint difference in MRR, 0.5965 to 0.4843, a gap of 0.1122, is larger than the sampling interval, whereas the difference between adjacent low weights is not. Exact skill matching therefore contributes usefully only at low weight, as a tiebreaker between occupations of comparable similarity, and not as a primary ranking signal. The reason is measurable: resume skill text and ESCO skill labels draw on different vocabularies, so exact string overlap fires infrequently on real resumes, and when it does fire it is often on a generic term. A skill-dominant setting allows one incidental match to outrank an occupation with strong semantic similarity and no exact match.

5.6 A harmful configuration caught by measurement

The re-ranking rule originally specified at Milestone 3 sorted on exact skill-match count with cosine similarity only as a tiebreak. When labelled data became available and the rule was parameterised and swept, that design was found to sit at the skill-dominant end of the range measured above — and to score MRR 0.4949 against 0.6437 for retrieval alone on the held-out split. The specified re-ranking step was actively degrading the ordering it was meant to improve.

It was replaced with the blended score at weight 0.02. This is recorded not as an incidental fix but as the clearest demonstration in the project of why the configuration surface had to be measured rather than reasoned about: the original rule was intuitively sensible and empirically harmful.

5.7 Prompt experiments

Experiment	Measured result	Decision
Broad skill search across the whole document	Skills F1 0.2268 → 0.1830; false positives 34 → 87	Rejected — duties were converted into skills
Atomic-skill splitting examples	Skills F1 0.6099 → 0.6115; precision 0.5811 → 0.5333	Rejected — negligible F1 gain, lower precision
Source-faithful boundaries	Skills F1 0.8198; Projects F1 0.8814	Accepted
Strict complete-line skills	Skills F1 0.8198 → 0.5607	Rejected — valid reference entities were omitted
Visual project boundaries	Projects F1 0.1724 → 0.4550	Retained — boundaries improved; named works still missed
Strict credential boundaries	Certifications F1 0.7472 → 0.6503	Rejected — global extraction loss exceeded the boundary correction

Table 5.2 — Resume Parsing prompt experiments and the decision taken on each.

Search scope mattered more than asking for completeness, and the two failure modes are symmetric: splitting grouped phrases raised recall and lowered precision, while preserving every printed line intact omitted tools embedded inside those lines. Stable aliases and unambiguous enumerations were therefore moved into deterministic comparison-time normalisation, and uncertain omissions and inventions were retained as model errors.

The strict-credential-boundaries result is worth dwelling on: it was a locally correct rule that was discarded on its net effect. It suppressed valid credentials alongside the invalid ones it was designed to catch, reducing full-run Certifications F1 from 0.7472 to 0.6503.

5.8 Cost and latency optimisation

Gradient-based optimisers, learning-rate schedules, and gradient clipping do not apply here, since no gradients are computed. The optimisation that did take place was of cost and latency.

Optimisation	Effect
Exactly two model calls per Job Discovery run	A naive per-item design would issue an estimated 45 or more calls per run. Deterministic extraction plus a batched judge reduces this to two regardless of how many jobs are found
Zero-LLM extraction	Structured fields are read from schema.org JobPosting JSON-LD or page metadata rather than from a model call. Faster, more reliable, and free per job
Persistent occupation index	Occupation embeddings are generated once offline, so no taxonomy embedding call is made at request time. A candidate query costs exactly one local embedding and one explanation call regardless of index size
Global deduplicated job store	Every posting URL reduces to a stable key. A posting is crawled, parsed, and embedded once, then reused by every subsequent search from any candidate until it goes stale or expires
Local embedding model	bge-base-en-v1.5 runs on CPU, so candidate and job embeddings cost nothing per call and are unaffected by API rate limits

6. Evaluation and Analysis

This section corresponds to Milestone 5, the evaluation and integration milestone. Results are reported with confidence intervals, and components that do not meet their acceptance criteria are reported as failing.

6.1 Metrics and why each was chosen

Metric	Definition and rationale
Precision, Recall, F1	TP/(TP+FP), TP/(TP+FN), 2TP/(2TP+FP+FN), micro-aggregated per section so sparse fields cannot dominate. Item-level true negatives are not defined, because the set of possible skills, employers and degrees is unbounded
Description similarity	Coverage plus TF-IDF cosine, used only for long free-text fields where faithful copy can differ character by character
Hit Rate at K	Fraction of profiles with an acceptable occupation in the top K — what the user actually sees
MRR	Mean reciprocal rank of the first acceptable occupation; the primary ordering measure
Precision@5, NDCG@10, Spearman	Job Matching ranking quality; Spearman is the most sensitive on a ten-item pool
Wilson score interval	95% interval on a binomial proportion, used because the normal approximation is unreliable at these rates and sample size
Coverage and crawl success	The two places the discovery pipeline can fail; crawl success is in practice the binding one

Metrics deliberately not used

- BLEU, ROUGE, BERTScore: no stage generates free text against a reference; the explanation model is checked by URI validation against retrieved evidence instead.
- ROC-AUC and PR-AUC: the match score is a deterministic decision-support value, not a calibrated probability.
- Confusion matrix: there is no small, fixed label set; ranking runs over 3,039 occupations. The informative equivalents are per-field confusion examples and the quantified miss decomposition in Section 6.5.
- Hallucination rate: prevented structurally by URI validation rather than measured statistically; the guard never needed to fire across the annotated runs.

6.2 Resume Parsing

Section	TP	FP	FN	Precision	Recall	F1	Result
Contact	139	3	2	0.98	0.99	0.98	Pass
Skills	1,190	395	128	0.75	0.90	0.82	Pass
Education	353	31	28	0.92	0.93	0.92	Pass
Experience	996	27	17	0.97	0.98	0.98	Pass
Projects	52	9	5	0.85	0.91	0.88	Pass
Certifications	133	73	17	0.65	0.89	0.75	Pass

Table 6.1 — Field-level Resume Parsing results over all 86 gold resumes. All six sections meet the 0.75 acceptance threshold.

Certifications pass on the unrounded value of 0.7472 and is the weakest section, driven by precision of 0.65: awards, honours and professional affiliations are returned as credentials. Skills carries the largest absolute false-positive count for a structural reason — grouped source phrases and individually annotated tools use different boundaries.

Experience descriptions are scored separately, because faithful copy of long text can differ character by character. Corresponding descriptions were present for 99.52 percent of matched

entries, mean TF-IDF cosine where present was 0.81, and the overall description score with missing positions counted as zero was 0.80 — above the threshold while still penalising omissions.

Against the baseline

Model	Contact	Skills	Education	Experience	Projects	Certifications
Gemma 4 31B (baseline)	0.79	0.53	0.77	0.79	0.45	0.51
Gemini 3.5 Flash (selected)	0.98	0.82	0.92	0.98	0.88	0.75

Table 6.2 — Section-level F1 for the selected parser against the baseline, on the same 86 resumes and the same corrected gold set.

The selected parser produced the higher F1 in every section, with the largest gains in the three weakest baseline sections: Projects +0.43, Skills +0.29, Certifications +0.24. Median latency was comparable (17.21 s against 19.03 s) and the selected model had the lower worst case (80.26 s against 126.51 s); the baseline also completed only 83 of 86 resumes against 86 of 86. The selection is justified by extraction quality, not by a claimed latency improvement.

6.3 Career Recommendation

Stage	MRR	Hit@1	Hit@3	Hit@5	Hit@7	Hit@20
Retrieval, Top-K 20	0.6110	0.4651	0.6977	0.8256	—	0.9070
Re-ranked	0.6121	0.4884	0.6860	0.8372	0.8488	—
Held-out split, 52 resumes	0.6359	0.5192	0.7500	0.8269	—	—

Table 6.3 — Career Recommendation results on the 86-record annotated gold set and the 52-resume held-out split.

The module produced a ranked list for every profile — 86 of 86 evaluated, zero failures. On the held-out split the selected configuration matches retrieval at ranks 1 and 3 and improves Hit@5 from 0.8077 to 0.8269, sitting at or above the full-set figures, which is consistent with a configuration that is not overfitted to the set it was chosen on.

Statistical treatment

Measure	Count	Rate	95% Wilson interval	Width
Hit@1, retrieval	40 / 86	0.4651	0.363 – 0.570	0.206
Hit@1, re-ranked	42 / 86	0.4884	0.386 – 0.592	0.207
Hit@5, retrieval	71 / 86	0.8256	0.732 – 0.891	0.159
Hit@5, re-ranked	72 / 86	0.8372	0.745 – 0.900	0.155
Hit@7, re-ranked	73 / 86	0.8488	0.758 – 0.909	0.151
Hit@20, retrieval	79 / 86	0.9186	0.841 – 0.960	0.119

Table 6.4 — Wilson score intervals. The intervals are wide, and that width is itself a finding.

6.4 The negative results

The re-ranking step's headline gain is not significant

The Hit@5 improvement from 0.8256 to 0.8372 is one profile: 71 against 72 of 86. A paired exact test over the discordant pairs gives $p = 1.00$. The re-ranking step is therefore not shown to improve Hit@5, and the earlier claim of a lift has been withdrawn.

What the evidence does support is the weaker claim that the step does not degrade the retrieval ordering — which the rule it replaced did, and substantially, as Section 5.6 records. Against that previously specified rule the held-out improvement is large and separable: MRR rises from 0.4949 to 0.6359 and Hit@5 from 0.6731 to 0.8269.

This caveat governs every configuration comparison in the project. At $n = 86$ the confidence interval on a rate near 0.83 is roughly plus or minus 0.08, so differences below about 0.08 are not separable. Sweeps are therefore read for the shape of their response curve rather than for the

winning value, and single-run differences smaller than the interval width are described as not separable rather than as improvements.

The batched judge stage does not work

Stage and variant	Composite	Schema	Calibration	Latency (ms)	Pass rate
Query gen. — boolean_operators, selected	0.5102	1.0000	0.8010	19	100%
Query gen. — remaining six variants	0.3602–0.4306	1.0000	0.8010	26–33	100%
Judge — comparative, best available	0.2750	0.5000	0.5000	21,572	0%
Judge — narrative	0.2060	0.5000	0.5000	28,082	0%
Judge — four remaining variants	–0.0668 to –0.1500	0.0000	0.0000	27,872–35,716	0%

Table 6.5 — Prompt sweeps for the two model-calling stages of Job Discovery.

The judge failures across all six variants trace to token size rather than instruction wording. The combined payload, full resume text, full job description, instructions and any few-shot context, exceeds the usable context budget for the judge call, producing truncation-driven and hard failures. Six instruction styles at a zero percent pass rate is consistent with a structural limit and not with a prompt-quality problem, which is why further rewording was abandoned.

A staged remedy is specified in order of effort: pre-flight token budgeting that counts tokens before the call and triggers a reduction step instead of sending and failing; structured extraction that passes skills, titles, years of experience, education and required qualifications rather than raw prose, which is the highest-leverage step because most of the token cost is text the judge does not need verbatim; and relevance-ordered truncation as a last resort. None of these is built, and the stage is reported as failing.

Job Matching's ablation is inconclusive

Neither the lexical nor the semantic signal can be shown necessary on this pool: removing the lexical signal costs 0.0067 on Precision@5 and removing the semantic signal changes it by –0.0026, both within noise. The hybrid score is retained on reasoning grounds — the lexical component catches exact tool and certification names that embeddings blur, the semantic component catches equivalent skills phrased differently — but that reasoning is not currently supported by measurement, and a ten-job pool is very likely too small to test it.

6.5 Failure decomposition

Because Hit@20 is measured on the same profiles as Hit@7, the misses decompose exactly rather than by estimate. Thirteen of 86 profiles had no acceptable occupation in the top seven.

Cause	Count	Share	Evidence
Truncation: present in top 20, lost at top 7	6	46%	Hit@20 0.9186 against Hit@7 0.8488; the re-ranker diagnostic records 6 dropped out of top 7
Never retrieved: absent from top 20	7	54%	86 – 79 = 7
of which taxonomy coverage gap	≈6	46%	Arts, Management and Network Security Engineer: 2 records each, all missed
of which other retrieval failure	≈1	8%	Remainder

Table 6.6 — Exact decomposition of the 13 misses at Top-7.

Truncation is the smaller half of the error and is already partly bought back. Moving the final list from five to seven raises Hit@7 to 0.8488 — 73 of 86, against Hit@5 of 0.8372, 72 of 86. One profile recovered for two extra slots. Six acceptable occupations still sit between ranks 8 and 20, so the remaining truncation loss is distributed rather than concentrated, and widening the list further

buys progressively less. This is worth stating plainly because the failure decomposition alone suggested a larger remedy than the measurement delivered.

Vocabulary mismatch is deliberately not listed as a separate row with a percentage. It is not separable at profile level: it degrades the skill-overlap signal across every profile rather than causing specific identifiable misses. Its measured effect is the monotonic decline of MRR from 0.5965 to 0.4843 as the skill weight rises. Assigning it a share of the 13 misses would double-count profiles already attributed to truncation or coverage, so it is quantified by its effect size instead.

6.6 The most consequential defect

One integration defect deserves recording in full, because it produced no error at any layer and would not have been caught by any surface check.

During the migration to the centralised vector store, the client's insertion helper wrote malformed embeddings. Content, metadata and row counts were all correct, so every conventional validation passed. It was detectable only by measuring the vectors themselves: a stored vector had an L2 norm of 0.5951 where this encoder always produces approximately 1.0, and comparing a stored vector against a fresh embedding of its own document text gave a cosine similarity of -0.0072 — a document orthogonal to itself. The user-visible symptom was retrieval nonsense.

The fix bypassed the helper to write the vector literal directly through the database driver, and a verification step now ships inside the ingestion script, so a rebuilt index is checked for vector norm and self-similarity rather than trusted because the insert succeeded. This is the single most important operational lesson from the project: for vector stores, a successful write is not evidence of a correct write.

6.7 Acceptance criteria and outcomes

Criterion	Outcome
Resume Parsing: per-field F1 ≥ 0.75 across all sections	Met — all six sections pass; lowest is Certifications at 0.7472
Resume Parsing: no schema violation on the development split	Met — schema validity moved from 0/5 to 5/5 on the smoke cohort after the structured-response correction
Career Rec.: re-ranking not below retrieval on primary measures	Met on every primary measure
Career Rec.: correct occupation within the initial retrieval window	Met — Hit@20 is 0.9186
Career Rec.: zero unrecoverable failures	Met — 86 of 86 profiles produced a ranked list
Job Matching: ranking quality $\geq$ default weights	Met — matches or exceeds the default composite and is an order of magnitude faster
Job Discovery: search coverage sufficient	Met — coverage 1.0 in every configuration
Integration: all modules over one application and database	Met

6.8 Computational performance

Stage	Measured	Notes
Resume parsing, per resume	19.03 s median, 21.79 s mean	All 86 resumes; maximum 80.26 s
Career Recommendation, per profile	1.5 to 1.8 s	132–158 s across 86 profiles against the hosted store
Job Matching ranking	226 ms	Against more than 4,000 ms for every rejected configuration
Job Discovery, API path	Within 2 minutes	Across 45 domains retrieved
Job Discovery, fallback crawl	2,703 ms	Against 26,604 ms for the slowest configuration tested

Query generation	19 ms	Range 19 to 33 ms across seven variants
Batched judge	21,572 ms median	Exceeds the 20-second ceiling; one case reached 178,664 ms
Occupation index build	Minutes on CPU	One offline pass over 3,039 documents

Centralising the vector store raised Career Recommendation evaluation time from 11.7 seconds to 132–158 seconds for the same 86 profiles. The cause is network round-trip time to the hosted region rather than additional computation, and it is a per-query cost rather than a scaling problem, since the index is queried once per profile regardless of its size.

6.9 Limitations of the evaluation

- 28 of the 86 gold records retain AI-drafted rather than fully source-verified annotations, so a residual accuracy ceiling applies to every metric computed over the full set.
- The Career Recommendation ground truth is a keyword-generated, hand-reviewed proxy rather than direct occupation labels, and several categories map to overlapping occupation sets.
- At $n = 86$ several reported differences — including the re-ranker's contribution — are not statistically separable, and are described as such throughout.
- The live-search evaluation improved in domain coverage but remains approximately one query per domain, so per-domain depth is shallow and the reported reliability figures are point estimates on very small samples.
- The skill-bonus weight sweep was run over all 86 records including the 52 held out, so for that one parameter selection and confirmation are not strictly separated; the held-out comparison for the selected value was run separately and is clean.
- No human evaluation of recommendation usefulness, explanation quality, or user satisfaction was conducted. This is the largest remaining methodological gap for a system whose output is advisory.
- The evaluation corpus is entirely single-page, English, and image-only, so scenarios outside that envelope cannot be measured against it without sourcing new data. Claims about production readiness are restricted to that envelope.

7. Deployment and Documentation

This section corresponds to Milestone 6, which turns the project from a set of evaluated components into a running, documented, reproducible product.

7.1 Value-added enhancement: the Feedback module

Milestone 6 required one enhancement outside the original scope, improving usability, explainability, reliability, or maintainability. The enhancement built is a Feedback module that allows a user to rate recommendations and stores that feedback for future improvement.

Why this enhancement

Every quality claim in this project rests on offline proxies. The evaluation measures whether an acceptable occupation is retrieved, not whether a candidate would act on it; URI validation proves the explanation cites only retrieved occupations but says nothing about whether the reasoning is useful to a reader. Section 6.9 names the absence of human evaluation as the largest remaining methodological gap. The Feedback module is the first piece of infrastructure that could eventually close it.

One design decision is worth highlighting: the feedback control is a floating button present on every screen, including for users who abandon the flow before reaching a final report. Those users are the most informative population and the one an end-of-flow survey structurally cannot reach.

Specification

Endpoint	Behaviour
POST /api/feedback	Rating 1–10, one or more closed-enum reasons, optional comment, optional profile_id
GET /api/feedback/mine	The current user's past submissions
GET /api/feedback/summary	Count, mean, distribution, reason counts — unauthenticated, aggregate counts only
GET /api/feedback/reasons	The twelve-value reason list, served from a backend constant so the UI can never drift out of sync

- The 1–10 rating bound is enforced twice: by Pydantic at the API boundary and by a database-level CheckConstraint. Validation that matters is not left to one layer.
- Reasons are a closed twelve-value enum served from the backend, so the interface cannot present an option the database will reject.
- profile_id is a nullable foreign key with ON DELETE SET NULL, so feedback survives profile deletion. The signal is retained even when the personal data it referred to is gone.
- All writes go through a FeedbackRepository; no raw SQL appears in the API layer.
- An import-linter contract declares that nothing depends on the feedback module. The module can fail, be disabled, or be removed without affecting any other path.

Honest limitations of the enhancement

Two limitations are documented in the module's own docstring rather than discovered later. The summary endpoint is unauthenticated — it exposes counts only, never comments or identifiers, but it needs an admin check before production deployment. And no model has been trained on the collected feedback; the data is being gathered for future improvement, and the module is presented as infrastructure rather than as a closed loop.

7.2 Deployment

The system is deployed on an AWS EC2 instance at <http://13.235.73.185:8080/>, active for the duration of the viva. Because the local embedding model (approximately 450 MB) plus the full application stack exceed what the AWS free tier provides, the instance runs on a paid t3.medium. If the hosted URL is unavailable, the local deployment produces identical output and is the intended fallback.

Component	What it is	Where it runs
Frontend	React 19 + TypeScript SPA, built with Vite, served by nginx	AWS EC2 (t3.medium), port 8080
Backend API	Single FastAPI application exposing all module routers	Same EC2 instance, port 8000 (proxied)
Database and vector store	Supabase PostgreSQL with pgvector, HNSW cosine index over 3,039 occupation vectors	Supabase managed hosting, ap-northeast-2
Embedding model	BAAI/bge-base-en-v1.5 (110M params)	Locally on CPU inside the backend process; cached from Hugging Face on first start
Generative models	Gemini 3.5 Flash, 3.5 Flash-Lite, 2.5 Flash-Lite	Hosted Google API — nothing is self-hosted
Search	SearXNG metasearch	Self-hosted Docker container, port 8888
Crawler	Crawl4AI (headless Chromium)	In-process, invoked by the Job Discovery fallback path

The repository ships a Dockerfile for the backend, a Dockerfile for the frontend, and a docker-compose.yml that starts the full stack together. For local development the uv path is substantially faster than a container build and is the documented default.

7.3 Documentation deliverables

Document	Audience	Contents
README.md	Anyone arriving at the repository	Project overview, quick start, architecture, evaluation summary, troubleshooting, known limitations
RUNNING.md	Someone setting the system up	Step-by-step first-time setup: prerequisites, environment configuration, SearXNG, migrations, backend, frontend, verification
docs/overview.md	Evaluators	Purpose, architecture summary with system diagram, deployed components
docs/technical_doc.md	Developers and future teams	The ten-section technical reference: environment, data pipeline, model architecture, training and evaluation summaries, inference pipeline, deployment, system design, error handling, reproducibility checklist
docs/LICENSES.md	Anyone reusing the work	Code license, dataset licenses, model sources and academic citations
docs/future_work.md	Maintainers	Known limitations, possible extensions, how to update the system, maintainer contacts

7.4 System design in the deployed form

Modularity, enforced rather than intended

Each module is a self-contained package with its own router, service, schemas, and internal implementation. Cross-module communication happens through versioned JSON contracts, not shared internals. This is enforced automatically: an import-linter contract set checks eleven architectural rules on every run, and the final branch reports eleven of eleven contracts kept, zero broken.

How the database and retriever interact

Offline and once, each occupation becomes one chunk — a semantic atomic unit rather than a sliding window, because the taxonomy is already structured and atomic. Chunks are embedded by the local encoder and written into a pgvector column with an HNSW index using the explicit cosine operator class. That explicit operator class is not incidental: measured against the default-distance index it raised retrieval Hit@5 from 0.7791 to 0.8256.

At request time the candidate profile is flattened into a query string and embedded by the same encoder. Using an identical model configuration for both offline indexing and online queries is a hard requirement, since vectors from different model versions are not comparable. pgvector runs an approximate-nearest-neighbour search and returns the top 20 occupations with their metadata. No embedding call is made against the taxonomy at request time.

Sequencing, and a race condition that had to be removed

Career Recommendation and Job Discovery originally ran concurrently under `asyncio.gather`. This produced a data race in which Job Discovery read a stale or missing recommendation. The two stages are now strictly sequential, and Job Discovery consumes the persisted recommendation run by profile identifier rather than an in-flight value. This is a correctness constraint, not a performance choice.

7.5 Error handling

The system's guiding rule is that a failure should degrade the response, not raise. Across the annotated evaluation this produced 86 of 86 profiles returning a ranked list, with zero unrecoverable failures.

Condition	Behaviour
Parser output fails schema validation or the completeness check	One fallback retry against a second model, then a partial profile with review flags rather than nothing

Candidate skills produce no overlap with any retrieved occupation	Similarity-only ranking, confidence reported as low, with an explicit message to the user
Explanation model names an occupation outside the retrieved set	Entry dropped by URI validation before the response is returned
Primary job API returns nothing or fails	Falls back to the search-and-crawl chain; an empty shortlist carries a no-matches status rather than raising
Crawl fails to render a posting	That URL is skipped; the run continues. One bad page does not fail the request
Database write fails after ranking	Error logged, shortlist still returned — a write failure must not block the response
Any pipeline node fails	Sets an error field that downstream nodes check before executing, so a failure short-circuits the run instead of raising

7.6 Reproducibility

Control	Value
Random seed	42, fixed across split assignment and sampling
Temperature	0 on every generative call
Output format	Schema-constrained JSON on every generative call, validated before downstream use
Provenance	Complete model and prompt fingerprints recorded per run
Database schema	Built from versioned Alembic revisions 0001 through 0005
Gold corrections	Stored as versioned overlays that preserve the original annotations
Grading	All evaluation grading is deterministic and rule-based, never model-graded

Run-to-run variance is nil by construction for the deterministic stages. The uncertainty that matters is sampling uncertainty over the 86 records, which the Wilson intervals in Table 6.4 quantify.

Check	Command	Result
Backend test suite	uv run pytest	74 passed
Architecture contracts	uv run lint-imports	11 kept, 0 broken
Frontend lint	npx eslint .	exit 0
Backend liveness	curl http://localhost:8000/health	{"status":"ok"}

The occupation index is pre-populated in the shared database, so no ingestion step is required for normal setup or evaluation; building it from scratch takes roughly an hour on CPU and is necessary only when pointing the application at a fresh, empty database. It should be noted honestly that no separate clean-machine rebuild beyond the documented steps was performed.

8. Conclusion and Future Work

8.1 What was built

All modules run inside a single application over a centralised database, completing the architecture proposed at Milestone 3. Resume Parsing has a complete field-level evaluation over all 86 gold resumes, with every section meeting the 0.75 threshold and the provider selection measured against an explicit baseline. Career Recommendation carries confidence intervals and a significance test, and its failure decomposition is exact rather than qualitative. The system is deployed, documented, and reproducible from a clone.

Component	Headline result	Status
Resume Parsing	All six sections ≥ 0.75 F1; Contact 0.98, Experience 0.98, Education 0.92, Projects 0.88, Skills 0.82, Certifications 0.75	Measured, passing
Career Recommendation	MRR 0.6121, Hit@1 0.4884, Hit@5 0.8372, Hit@7 0.8488, Hit@20 0.9186, 0 failures in 86	Measured; re-ranker gain not significant

Career Recommendation (held-out)	MRR 0.6359, Hit@5 0.8269 on 52 unseen resumes	Measured, generalises
Job Matching ranking	Precision@5 0.8486, NDCG@10 0.9795, Spearman 0.8767 at 226 ms	Measured; efficiency result, quality flat
Job Discovery	Coverage 1.0; API layer validated across 45 domains; fallback crawl 0.50	Measured; reliability below acceptance
Query generation	Schema 1.0000, diversity 0.8010, pass rate 100%	Measured, passing
Batched judge	Schema 0.5000, calibration 0.5000, pass rate 0%	Measured, failing; root cause identified
Feedback module	11 of 11 architecture contracts kept; 74 of 74 tests passed	Measured, passing
Integration and deployment	All modules over one application and database; deployed on AWS EC2	Complete

8.2 What was learned

Four lessons from this project generalise beyond it.

Measurement caught a configuration that reasoning had endorsed

The re-ranking rule specified at Milestone 3 — sort on exact skill-match count, break ties on similarity — was intuitively sensible and empirically harmful, scoring MRR 0.4949 against 0.6437 for retrieval alone. It was only found once labelled data made the sweep possible. The general lesson is that a plausible heuristic in a ranking system is a hypothesis, not a design.

A successful write is not a correct write

The vector corruption defect passed every conventional check — correct content, correct metadata, correct row counts — and was detectable only by measuring vector norms and self-similarity. Any system that stores derived numerical artefacts needs a verification step that inspects the artefact itself, not the operation that produced it.

Statistical honesty changes what can be claimed

At $n = 86$, a 0.0116 improvement in Hit@5 looks like progress and is, on a paired exact test, one profile at $p = 1.00$. Reporting the interval alongside the point estimate is what turned an apparent success into an accurately stated null result. Several claims in earlier milestones were withdrawn on this basis.

Constraints shaped the architecture more than preferences did

The absence of labelled data forced retrieval over classification. The absence of a GPU and the presence of free-tier quota forced the local encoder, the offline index, the batched judge, and zero-LLM extraction. Almost every design decision in this system traces to a constraint rather than to a preference and documenting that trace is more useful to a future reader than presenting the outcome as a choice.

8.3 Known gaps carried forward

Gap	Evidence
The batched judge stage does not produce usable output	0% pass rate across six variants; root cause diagnosed as token overflow; remedy specified but unbuilt
Crawl success remains 0.50	The API layer reduces how often the crawl path is reached but does not repair it, and the API does not serve niche domains — precisely the case that falls through to the crawl
No system-level discovery reliability figure exists	The 0.50 figure measures the crawl layer in isolation; the two-layer pipeline has not been swept end to end
No human evaluation	No usefulness, explanation-quality, or satisfaction study was conducted

Gender-bias channel untested	The gold set carries no gender annotation and no counterfactual test was constructed, though names reach the embedding stage
Encoder truncation unquantified	The encoder caps at 512 tokens against occupation documents reaching roughly 1,300; the affected subset has not been isolated
Taxonomy coverage gaps	Arts, Management, and Network Security Engineer have no counterpart in the taxonomy and all their records miss
No load or concurrency testing	Only single- and double-user latency has been measured
Feedback collected but unused	No model has been trained on it; stated as an intentional scope limitation
Untested outside a narrow envelope	The corpus is single-page, English, and image-only; multilingual and multi-page behaviour is unknown, not merely unmeasured

8.4 Future work

Each option below follows from a specific measurement in this report and is stated with its cost rather than as a commitment. They are ordered by the size of the measured problem they address.

1. Construct a gender counterfactual test set: matched profile pairs with substituted names and gendered terms, measuring rank shift on identical qualifications.
2. Close the vocabulary gap, in ascending order of cost: embed the 13,939 skill labels for semantic matching, add a curated alias table, or run a cross-encoder re-ranker over the top twenty.
3. Add throughput and cost instrumentation. Tokens are already recorded per call, so cost per resume is derivable from the existing ledger; a concurrent load harness is the missing piece.
4. Extend the corpus beyond English. This requires sourcing new annotated data and is the largest single cost on this list, but it is the only way to make claims outside the current envelope.
5. Use the collected feedback to recalibrate the values still labelled heuristic: shortlist size, hybrid and judge weights, essential and optional skill weights, and the latency term in the composite.

8.5 Closing statement

The state of the system at this final milestone is that its measured components are measured properly and at stated confidence, its unmeasured components are named as unmeasured rather than assumed adequate, and its one failing component is reported as failing with a diagnosed but unbuilt fix.

The Resume Parsing and Career Recommendation path is ready to be used as decision support, which is exactly how it is presented to the user: a short, ranked list with confidence bands and an explanation grounded in retrieved evidence. It is not ready to be used as an automated decision, and it is not presented as one. Job Discovery is architecturally complete and functionally incomplete — its judge stage has a diagnosed but unbuilt fix, and its crawl fallback remains unreliable.

Three gaps are carried forward without mitigation: no human evaluation, no gender-bias measurement, and no testing outside a single-page English image-only envelope. Each is a real limit on what this work can support, and each is stated rather than absorbed into a favourable summary.

9. References

9.1 Datasets and taxonomies

European Commission. ESCO (European Skills, Competences, Qualifications and Occupations) classification, version 1.2.1. Licensed under CC BY 4.0. <https://esco.ec.europa.eu/>

Kaggle. Resume Data (PDF), dataset hadikp/resume-data-pdf.

Kaggle. Candidate Job Role Dataset.

Kaggle. LinkedIn Job Postings Dataset.

9.2 Models

Xiao, S., Liu, Z., Zhang, P., and Muennighoff, N. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597. Model: BAAI/bge-base-en-v1.5, MIT License. Google. Gemini API documentation and model family (Gemini 3.5 Flash, Gemini 3.5 Flash-Lite, Gemini 2.5 Flash-Lite).

Google. Gemma model family (Gemma 4 31B), used as the Resume Parsing evaluation baseline.

9.3 Methods

Robertson, S. E., and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389.

Malkov, Y. A., and Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence.

Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association, 22(158), 209–212.

9.4 Software

FastAPI, SQLAlchemy, Alembic, pgvector, sentence-transformers, rank_bm25, LangChain, LangGraph, PyMuPDF, Crawl4AI, SearXNG, React, Vite, pytest, import-linter. Full pinned versions in backend/uv.lock and frontend/package-lock.json; licenses recorded in docs/LICENSES.md.

10. Team Consent

Name	Roll Number	Initial
Gaurav Kumar	22f1001105	GK
Dev Gupta	22f2000888	DG
Abhinav Ohri	24f1002064	AO
Pranav N	22f2000117	PN
Praveena N	22f3001454	PN

Appendix

Appendix A — Repository structure

Path	Purpose
backend/main.py	Application entrypoint
backend/src/app.py	FastAPI application factory, CORS middleware, router registration
backend/src/core/config.py	All environment variables and derived settings, read once at startup
backend/src/core/db.py	Async SQLAlchemy engine, session factory, connect-args
backend/src/resume_parsing/	Router, service, schemas, internal pipeline and providers
backend/src/career_recommendation/	API, service, retrieval, re-ranker, store, ingestion, profile mapper
backend/src/career_recommendation/evaluation/	evaluate_gold.py, gold.jsonl, recorded results, category-to-occupation mapping
backend/src/job_discovery_matching/	API, service, store, seven-node LangGraph pipeline, internal services
backend/src/career_report/	Router, service, HTML generator, PDF renderer
backend/src/feedback/	New in Milestone 6: API, schemas, models, FeedbackRepository
backend/src/cv_review/	Recruiter-style CV critique add-on; read-only
backend/src/services/llm_client.py	Shared embedding client and generative-model wrappers
backend/alembic/versions/	Migrations 0001 through 0005
backend/experiments/	Prompt sweep scripts, per-resume evaluation CSVs, charts
backend/tests/	74 pytest tests
frontend/src/	React 19 + TypeScript: resume parsing, career guidance, feedback
data/	Dataset layout and sample data; raw datasets gitignored
docs/	Milestone reports, technical documentation, architecture diagram, contribution tracker
notebooks/	EDA and preprocessing notebooks
RUNNING.md, README.md, docker-compose.yml, .env.example	Setup, overview, container stack, environment template

Appendix B — API endpoints

Endpoint	Method	Purpose
/health	GET	Liveness check
/docs	GET	Interactive API documentation
/api/resume-parsing/resumes	POST	Upload and parse a resume, streaming progress
/api/career/recommend	POST	Ranked occupation recommendations with grounded explanations
/api/jobs/search	POST	Seven-stage job discovery and matching
/api/feedback	POST	Submit a rating, reasons, and optional comment
/api/feedback/mine	GET	Current user's past submissions
/api/feedback/summary	GET	Aggregate counts only
/api/feedback/reasons	GET	The twelve-value reason enum

Appendix C — Worked example of Career Recommendation output

Deterministic ranking before the explanation call, for a data-analyst profile, showing the two-part score:

Occupation	Matched essential skills	Similarity
data analyst	Python, SQL, data visualisation, statistics	high
data scientist	Python, SQL, statistics	high
statistical assistant	statistics	medium
financial analyst	statistics, SQL	medium
database developer	SQL	medium

The confidence bands track the deterministic evidence rather than being asserted by the model: five matched skills yield high confidence, one matched skill yields low. The low-confidence entry states what is missing rather than overselling the match. Every returned occupation URI in this run passed validation against the retrieved set, so no entry was dropped by the anti-hallucination guard.

Appendix D — Integration verification record

The merged path was exercised end to end on a real image-only resume from the corpus. The chain ran without manual intervention.

Stage	Result
Parsing	Valid on the first attempt: fallback not invoked, no review flags, descriptions copied verbatim, personal identifiers correctly excluded
Normalisation	Six of seven skill bullets retained, the seventh correctly removed as a near-duplicate
Recommendation	Top 7: agricultural engineer, agricultural equipment design engineer, test engineer, agricultural machinery technician, agronomist, agricultural researcher, agricultural technician
Ranking accuracy	The candidate's actual occupation returned at rank 1, with all seven recommendations in the correct domain
Confidence handling	The resume writes skills as full sentences, so the deterministic overlap signal had nothing to match against short taxonomy labels. The system detected this, ranked on semantic evidence alone, returned the correct occupation first, and reported low confidence with an explicit statement of why
Job search	Target roles seeded from the stored recommendation run without manual input; search, ranking and shortlist stages executed inside the same request

Appendix E — Contribution summary

Name	Roll Number	Primary contribution
Gaurav Kumar	22f1001105	Career Recommendation module (embedding, retrieval, deterministic re-ranking, grounded explanation); milestone report and presentation preparation; frontend corrections; pipeline integration; Feedback module; CV Review module; end-to-end human testing
Dev Gupta	22f2000888	Career Recommendation module (embedding, retrieval, deterministic re-ranking, grounded explanation); GitHub management; deployment; end-to-end human testing
Abhinav Ohri	24f1002064	Resume Parsing module (post-processing, multi-page merge); frontend development; Career Report preparation; evaluation harness support
Pranav N	22f2000117	Job Discovery and Matching module (LangGraph pipeline, search and crawl integration, hybrid ranking, matching and judge integration); evaluation harness support
Praveena N	22f3001454	Job Discovery and Matching module (LangGraph pipeline, search and crawl integration, hybrid ranking, matching and judge integration); evaluation harness support; contribution tracker preparation